

Функции для работы с DOM элементом на JavaScript

Сейчас мы научимся делать вспомогательные функции, выполняющие какие-либо операции с DOM. Например, давайте сделаем функцию, которая первым параметром будет принимать id элемента, а вторым - текст элемента, и будет устанавливать этому элементу новый текст.

Вот описанная функция:

```
1 function setText(id, text) {  
2   let elem = document.getElementById(id);  
3   elem.textContent = text;  
4 }
```

Давайте опробуем ее работу. Пусть у нас есть два абзаца:

```
1 <p id="elem1"></p>  
2 <p id="elem2"></p>
```

Давайте поменяем текст этим абзацам, воспользовавшись созданной нами функцией:

```
1 setText('elem1', 'text1');  
2 setText('elem2', 'text2');
```



№1

©jsPmFDDEF

Модифицируйте созданную мною функцию так, чтобы параметром она принимала не id элемента, а произвольный CSS селектор.



№2

©jsPmFDDEF

Сделайте функцию setAttr, которая будет менять атрибут DOM элементу. Пусть первым параметром функция принимает селектор элемента, вторым - имя атрибута, а третьим - его новое значение.

Функции для работы с группой DOM элементов

Давайте теперь напишем функцию, которая параметром будет принимать селектор группы элементов и их новый текст. Пусть эта функция устанавливает всем подпадающим под селектор элементам новый текст.

Реализуем описанную функцию:

```
1 | function setText(selector, text) {  
2 |     let elems = document.querySelectorAll(selector);  
3 |  
4 |     for (let elem of elems) {  
5 |         elem.textContent = text;  
6 |     }  
7 | }
```

Давайте проверим ее на группе следующих элементов:

```
1 | <p class="elem"></p>  
2 | <p class="elem"></p>  
3 | <p class="elem"></p>
```

Установим всем элементам с классом `elem` новый текст:

```
1 | setText('.elem', 'text');
```



№ 1

©jsPmFDEGF

Сделайте функцию `appendText`, которая первым параметром будет принимать селектор, а вторым - текст. Сделайте так, чтобы данная функция добавляла текст в конец переданных элементов.

Передача коллбэка для работы с DOM на JavaScript

Пусть у нас есть некоторая группа DOM элементов:

```
1 <p class="elem">1</p>
2 <p class="elem">2</p>
3 <p class="elem">3</p>
4 <p class="elem">4</p>
5 <p class="elem">5</p>
```

Давайте сделаем функцию `forEach`, которая первым параметром будет принимать селектор группы элементов, а вторым параметром - функцию-коллбэк, которая применится по очереди к каждому из найденных элементов:

```
1 forEach('.elem', function() {
2   // функция применится к каждому элементу
3 });
```

Пусть элементы, попавшие под селектор, по очереди попадают в первый параметр коллбэка:

```
1 forEach('.elem', function(elem) {
2   console.log(elem); // выведет по очереди
                       найденные элементы
3 });
```

Давайте с помощью нашей функции найдем все элементы с классом `elem` и для каждого найденного элемента применим коллбэк, который возведет в квадрат текст каждого элемента:

```
1 forEach('.elem', function(elem) {
2   elem.textContent = elem.textContent ** 2;
3 });
```

Давайте теперь напишем реализацию задуманной нами функции `forEach`:

```
1 function forEach(selector, func) {
2   let elems = document.querySelectorAll(selector);
3
4   for (let elem of elems) {
5     func(elem);
6   }
7 }
```

№1

© jsPmFDCP

Даны абзацы. С помощью созданной нами функции `forEach` для каждого абзаца добавьте в конец его текста восклицательный знак.

Передача порядкового номера в коллбэк на JavaScript

Давайте сделаем так, чтобы, если в нашей функции-коллбэке написан второй параметр, в него пусть попадает порядковый номер элемента в найденном наборе элементов:

```
1 | forEach('.elem', function(elem, index) {  
2 |   console.log(elem); // выведет элемент  
3 |   console.log(index); // выведет порядковый номер элемента  
4 | });
```

С помощью этого параметра мы можем, к примеру, добавить в конец каждому элементу его порядковый номер:

```
1 | forEach('.elem', function(elem, index) {  
2 |   elem.textContent = elem.textContent + index;  
3 | });
```

Давайте переделаем код нашей функции forEach для осуществления описанного:

```
1 | function forEach(selector, func) {  
2 |   let elems = document.querySelectorAll(selector);  
3 |  
4 |   for (let i = 0; i < elems.length; i++) {  
5 |     func(elems[i], i);  
6 |   }  
7 | }
```



№ 1

©jsPmFDcSNP

Даны абзацы. С помощью созданной нами функции forEach для каждого абзаца добавьте ему в начало его порядковый номер.

Передача DOM элемента параметром функции

В предыдущих уроках мы передавали в наши функции селекторы элементов, и наши функции сами получали ссылки на эти элементы внутри своего кода. Существует и другой подход: в функции можно передавать уже полученные ранее ссылки на элементы.

Давайте посмотрим на примере. Пусть у нас есть следующий код:

```
1 <p id="elem1"></p>
2 <p id="elem2"></p>
```

Давайте сделаем функцию `setText`, которая первым параметром будет принимать ссылку на DOM элемент, а вторым - текст элемента:

```
1 function setText(elem, text){
2   elem.textContent = text;
3 }
```

Используем созданную нами функцию для установки текста наших абзацев:

```
1 let elem1 = document.getElementById('elem1');
2 setText(elem1, 'text1');
3
4 let elem2 = document.getElementById('elem2');
5 setText(elem2, 'text2');
```

№1

© jsPmFDPEAP

Сделайте функцию `appendText`, которая первым параметром будет принимать DOM элемент, а вторым - текст. Сделайте так, чтобы данная функция добавляла текст в конец этого элемента.

№2

© jsPmFDPEAP

Даны абзацы. Получите их, переберите циклом и каждому добавьте в конец '!' с помощью функции `appendText`, сделанной в предыдущей задаче.

№3

© jsPmFDPEAP

Сделайте функцию `setValue`, которая первым параметром будет принимать ссылку на инпут, а вторым - текст. Сделайте так, чтобы данная функция устанавливала переданный текст в `value` инпута.

Передача группы DOM элементов параметрами функций

Пусть теперь наша функция `setText` первым параметром будем принимать ссылку не на один элемент, а сразу набор элементов:

```
1 function setText(elems, text) {  
2   for (let elem of elems) {  
3     elem.textContent = text;  
4   }  
5 }
```

Давайте опробуем нашу функцию на практике. Пусть у нас есть следующие абзацы:

```
1 <p class="elem"></p>  
2 <p class="elem"></p>  
3 <p class="elem"></p>  
4 <p class="elem"></p>  
5 <p class="elem"></p>
```

Давайте с помощью нашей функции установим всем этим абзацам какой-нибудь текст:

```
1 let elems = document.querySelectorAll('.elem');  
2 setText(elems, 'text');
```

№ 1

©jsPmFDPGAP

Сделайте функцию `appendText`, которая первым параметром будет принимать массив DOM элементов, а вторым - текст. Сделайте так, чтобы данная функция добавляла текст в конец переданных элементов.

№ 2

©jsPmFDPGAP

Сделайте функцию `appendElem`, которая первым параметром будет принимать ссылку на DOM объект, в котором лежит тег `ul`, а вторым - текст. Сделайте так, чтобы данная функция создавала новую `li` с переданным текстом и добавляла ее в конец переданного тега `ul`.

Функция для создания HTML таблиц на JavaScript

Давайте теперь сделаем функцию `createTable`, которая будет создавать таблицу заданного размера и добавлять ее в конец заданного элемента.

Пусть первым параметром наша функция принимает количество рядов, вторым параметром - количество колонок, а третьим - ссылку на DOM элемент, внутри которого будет создаваться наша таблица.

Давайте посмотрим, как мы будем пользоваться описанной функцией, когда она будет создана. Пусть, к примеру, у нас дан вот такой див:

```
1 | <div id="elem"></div>
```

Давайте сделаем внутри этого дива таблицу 3 на 4:

```
1 | let div = document.querySelector('#elem');
2 | createTable(3, 4, div);
```

Пусть теперь у нас даны два дива:

```
1 | <div id="elem1"></div>
2 | <div id="elem2"></div>
```

Давайте сделаем свою таблицу в каждом из этих дивов:

```
1 | let div1 = document.querySelector('#elem1');
2 | createTable(3, 4, div1);
3 |
4 | let div2 = document.querySelector('#elem2');
5 | createTable(5, 6, div2);
```

Для того, чтобы создаваемые таблицы сразу были видны, можно добавить какой-нибудь CSS, например, такой:

```
1 | td {
2 |   width: 50px;
3 |   height: 50px;
4 |   border: 1px solid black;
5 | }
```

Вот заготовка описанной функции:

```
1 function createTable(rows, cols, parent) {  
2   let table = document.createElement('table');  
3  
4   // тут создадим таблицу с rows рядами и cols колонками  
5   for () {  
6     for () {  
7  
8     }  
9   }  
10  
11   parent.appendChild(table);  
12 }
```

Допишите код представленной выше заготовки функции. Потестируйте работу готовой функции.

Возврат таблицы из функции на JavaScript

В предыдущем уроке мы сделали функцию `createTable`. Давайте модифицируем эту функцию так, чтобы она не добавляла таблицу в какой-то элемент, а просто возвращала ее через `return`.

То есть приведенный код предыдущего урока преобразуется вот в такой:

```
1 let div = document.querySelector('#elem');
2 let table = createTable(3, 4);
3 div.appendChild(table);
```

Можно переписать короче:

```
1 let div = document.querySelector('#elem');
2 div.appendChild(createTable(3, 4));
```

Получение ссылки на таблицу может понадобиться для того, чтобы что-то сделать с созданной таблицей. К примеру, давайте покрасим ее цвет текста в красный:

```
1 let div = document.querySelector('#elem');
2
3 let table = createTable(3, 4);
4 table.style.color = 'red';
5
6 div.appendChild(table);
```

№1

© jsPmFDRT

Переделайте вашу функцию `createTable` в соответствии с описанным в теории.

№2

© jsPmFDRT

Пусть у нас есть вот такой див с абзацами:

```
1 <div id="elem">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>
```

С помощью функции `createTable` создайте новую таблицу, а затем вставьте ее в конец дива.

Функция для создания таблицы из массива

В предыдущих уроках мы сделали функцию `createTable`, которая создает таблицу заданного размера. Давайте теперь сделаем функцию `createTableByArr`, которая параметром будет принимать двухмерный массив и строить на его основе таблицу.

То есть приведенный код предыдущего урока преобразуется вот в такой:

```
1 let div = document.querySelector('#elem');
2
3 let arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
4 let table = createTableByArr(arr);
5
6 div.appendChild(table);
```

В результате выполнения этого кода должна получиться следующая таблица:

```
1 <div id="elem">
2   <table>
3     <tr>
4       <td>1</td>
5       <td>2</td>
6       <td>3</td>
7     </tr>
8     <tr>
9       <td>4</td>
10      <td>5</td>
11      <td>6</td>
12    </tr>
13    <tr>
14      <td>7</td>
15      <td>8</td>
16      <td>9</td>
17    </tr>
18  </table>
19 </div>
```

№ 1

©jsPmFDTCFA

Реализуйте описанную функцию. Проверьте ее работу.